

FlexiServe: Enabling LLM Pipeline Reconfiguration with Elastic KV Cache Resizing

Abstract

With the rapid advancement of large language models (LLMs), efficiently serving LLM inference under limited GPU resources has become a critical challenge. Recently, an increasing number of studies have explored applying serverless computing paradigms to LLM serving in order to maximize resource utilization. However, LLM inference workloads are highly diverse, and modern GPU clusters are inherently heterogeneous, making it necessary to dynamically adjust deployment configurations online to better adapt to the elastic and dynamic nature of serverless environments. At the same time, enabling such online reconfiguration is particularly challenging due to the stateful nature of LLM inference and the massive size of model parameters. In this paper, we propose a dynamic pipeline reconfiguration approach that enables online adjustment of pipeline configurations while minimizing service downtime and performance degradation. Our method allows the system to select the optimal pipeline configuration in response to changing workloads. Experimental results on heterogeneous GPU platforms, including NVIDIA A100 and L40s, demonstrate that our migration mechanism incurs less than 50 ms of service downtime, while introducing under 10% overhead on both time-to-first-token (TTFT) and time-per-output-token (TPOT).

Keywords: Cloud-edge continuum, microservice, scheduling

1 Introduction

Large language models (LLMs) have advanced rapidly in recent years, driven by improvements in model architectures, training algorithms, and the availability of large-scale data and compute resources. This progress has enabled LLMs to serve as a core foundation for a wide range of applications, including AI-assisted programming, agentic systems, conversational interfaces, and domain-specific intelligent assistants. As these applications increasingly move from experimental use to production deployment, the efficiency and scalability of LLM serving systems have become critical system-level concerns.

Unlike traditional AI workloads, LLM serving places sustained and intensive demands on hardware accelerators such as GPUs or TPUs. Consequently, efficient utilization of available GPU resources is essential. Prior works have explored deploying LLMs across multiple GPUs using heterogeneous devices, to aggregate compute capacity and reduce idle time. Pipeline Parallelism (PP) which is model distribution approach to allocate a partial of sequence of model layer in multiple

GPUs is used to spread computational load in heterogeneous gpus.

Existing LLM inference systems typically rely on offline profiling to determine a fixed PP configuration and assume that the layer-to-GPU mapping remains unchanged during inference. However, LLM serving systems must handle highly diverse workloads that differ in execution patterns—such as interactive chatbots, agentic workflows, and offline batch processing—as well as in performance objectives. In heterogeneous GPU environments, we observe that the optimal PP configuration varies substantially across workload patterns. This observation implies that to effectively support diverse workloads and serving requirements, LLM serving systems require mechanisms to dynamically reconfigure the number of model layers assigned to each GPU under pipeline parallelism.

However, Enabling efficient reconfiguration of pipeline parallelism at runtime is non-trivial. Under current approaches, redistributing model layers across GPUs requires pausing inference to reload model weights and reinitialize KV caches, which can severely degrade inference performance. This challenge is further compounded by the memory management strategies used in state-of-the-art inference frameworks such as vLLM and SGLang, which adopt PageAttention as KV cache allocation strategy to reduce fragmentation. While effective under static configurations, PageAttention relies on layer-level pre-allocation of KV cache memory. During pipeline reconfiguration, adapting to a new PP configuration requires resizing the KV cache accordingly, yet the pre-allocated nature of PageAttention makes such resizing infeasible.

Together, these limitations hinder dynamic pipeline reconfiguration and motivate the need for system support that enables flexible layer migration and KV cache adaptation with minimal disruption to inference performance. Based on the insight that static pipeline-parallel configurations are insufficient for LLM deployment, we present **FlexiServe**, a dynamic inference architecture that enables online adjustment of layer placement across GPUs under pipeline parallelism.

FlexiServe adopts a top-down design spanning three coordinated levels. At the *model level*, we introduce a placement algorithm that, given the current and target pipeline configurations, evaluates migration feasibility based on KV cache occupancy and GPU memory availability, and generates explicit migration instructions for each GPU worker. At the *worker level*, a *Layer Migrator* runs on each GPU and performs asynchronous model parameter loading, KV cache

resizing, and KV cache migration during PP reconfiguration. At the *kernel level*, to support efficient KV cache resizing, we employ a block-level KV cache pre-allocation strategy that enables dynamic expansion or shrinking of pre-allocated KV cache space. Together, these components allow FlexiServe to reconfigure pipeline parallelism at runtime while minimizing disruption to ongoing inference.

Our **main contributions** are as follows:

- We identify that in heterogeneous GPU deployment environments, no single pipeline-parallel (PP) configuration can efficiently adapt to all LLM workloads due to variations in workload characteristics and performance objectives. This observation motivates the need for an LLM serving framework that supports efficient PP reconfiguration.
- We present **FlexiServe**, a unified LLM serving framework that enables dynamic PP reconfiguration. FlexiServe adopts a top-down system design that jointly optimizes from model-level to the attention kernel-level, minimizing inference overhead and service downtime during reconfiguration.
- We conduct comprehensive experiments to isolate and quantify the impact of each optimization used during pipeline reconfiguration. Our results show that with FlexiServe, inference interruption caused by KV cache resizing and layer synchronization is limited to within 10 ms, representing a substantial improvement over prior approaches that incur service pauses on the order of seconds.

2 Background

2.1 LLM Inference

Unlike conventional AI workloads, LLMs inference operate on variable-length inputs and produce variable-length outputs. Prompt processing is referred to as the *prefill* stage: the model consumes the prompt and computes activations for all prompt tokens in parallel, and this stage is typically compute-bound. After prefill, the model generates the next most likely token and feeds it back as input, producing an output sequence autoregressively; this process is the *decode* stage. During decode, the model generates one token at a time and is often memory-bound due to repeated reads and writes of cached attention states and the low arithmetic intensity per step.

During inference, LLMs materialize key-value (KV) caches to accelerate subsequent token generation. As context lengths and request concurrency increase, KV cache memory often becomes the dominant memory footprint. *PagedAttention* improves KV cache utilization by managing the cache in fixed-size blocks (pages) and allocating these blocks to requests on demand, thereby reducing memory fragmentation.

LLM serving performance is typically characterized by three metrics. *Throughput* measures steady-state generation

capacity (e.g., output tokens per second or requests per second) and is central for batch-style workloads. *Time-to-first-token (TTFT)* captures the latency from request arrival to the first generated token. *Time-per-output-token (TPOT)* measures the average latency per generated token after the first token. Different workload patterns emphasize different metrics, creating inherent trade-offs in serving system design.

2.2 Deployment of LLM Models

We refer to the assignment of model layers to GPUs under pipeline parallelism as a *pipeline-parallel configuration* (PP configuration). Pipeline parallelism typically assigns an equal number of layers to each GPU. In heterogeneous environments, however, GPUs differ in compute throughput, memory bandwidth, and memory capacity, and the layer partition that satisfies a given service-level objective (SLO) is often inherently unbalanced. As a result, effective PP configuration in heterogeneous GPU setups may require *uneven* layer-to-GPU assignments to align layer placement with device capabilities and performance goals.

LLMs are typically deployed on GPUs. In homogeneous GPU clusters, devices are often tightly integrated within a single node with high-bandwidth interconnects, making TP an effective choice. In contrast, when inference spans multiple nodes or when GPUs are connected by lower-bandwidth links as is common in heterogeneous deployments, pipeline parallelism (PP) is generally more suitable. This work therefore focuses on PP-based LLM serving.

We refer to the assignment of model layers to GPUs under pipeline parallelism as a *pipeline-parallel configuration* (PP configuration). In homogeneous clusters, PP commonly assigns an equal number of layers to each GPU. In heterogeneous deployments, however, GPUs differ substantially in hardware characteristics and this often results in an uneven deployment of model layers. Table X compares representative devices: A100 is a high-end accelerator designed for AI workloads and equipped with HBM, providing significantly higher memory bandwidth and larger memory capacity than L40s; in contrast, L40s is based on a newer architecture and can offer higher peak compute throughput in some regimes.

3 Motivation

When deploying LLMs on heterogeneous GPUs, existing systems typically profile devices to quantify relative capability in compute throughput, memory bandwidth, and memory capacity, and then derive an SLO-aware pipeline-parallel (PP) configuration that is optimized for a target workload. This approach is inherently workload-specific: we find that in heterogeneous settings, the optimal PP configuration can shift significantly with workload properties such as input/output lengths and request rate. In particular, different workloads stress different phases of inference (compute-bound prefill vs.

memory-bound decode), and a fixed layer partition cannot remain optimal across these regimes.

Figure X illustrates this effect on a two-GPU setup (A100 + L40s). For requests with long inputs (prefill-dominated), assigning more layers to L40s yields lower TTFT/TPOT; for requests with long outputs (decode-dominated), allocating more layers to A100 improves TTFT/TPOT. A plausible explanation is that A100's higher memory bandwidth better serves decode-intensive execution, whereas L40s can be more effective for compute-intensive prefill.

Motivated by this observation, we develop **FlexiServe**, which dynamically reconfigures PP layer placement at run-time to match the current workload. As shown in the third plot of Figure X, when the workload shifts mid-run (from short-input to long-input requests), FlexiServe switches PP configurations online and outperforms static partitions. Achieving this benefit requires a top-down reconfiguration design that minimizes disruption to throughput, TTFT, and TPOT during configuration changes.

Figure X (third plot) demonstrates this capability in a workload-shift experiment: the request mix changes mid-run from short-input to long-input prompts, altering the dominant inference bottleneck. In response, FlexiServe switches to a more suitable PP configuration online and achieves lower TTFT/TPOT without sacrificing steady-state throughput. To enable such adaptation, FlexiServe adopts a top-down reconfiguration design with the explicit goal of minimizing downtime and bounding the impact of reconfiguration on throughput, TTFT, and TPOT.

4 Overview of FlexiServe

FlexiServe is an LLM inference framework that enables run-time reconfiguration of pipeline-parallel (PP) layer placement. At the *model level*, FlexiServe designates one GPU node as the coordinator, which hosts the inference scheduler and a centralized *Reconfiguration Coordinator*. Given the current and target PP configurations, the coordinator runs a placement algorithm to (1) evaluate reconfiguration feasibility based on GPU memory availability and KV cache state, and (2) generate an explicit reconfiguration plan that translates the target configuration into a sequence of coordinated actions, including KV cache resizing and migration, asynchronous weight loading, and final state synchronization.

At the *GPU level*, each device runs an *inference worker* dedicated to executing model inference, alongside a local *Reconfiguration Worker* responsible for carrying out reconfiguration actions. Each reconfiguration worker comprises three components. (1) A *KV cache synchronizer* migrates KV states between source and destination layers during reconfiguration while preserving correctness under ongoing inference. To avoid perturbing inference communication,

KV transfers are executed in a dedicated NCCL group isolated from the inference group, with explicit handling of deadlock risks arising from concurrent collective operations. (2) A *weight loader* asynchronously stages newly assigned layer weights onto the target GPU. To bound interference with inference, weights are loaded in rate-limited chunks, trading slightly longer migration time for reduced performance impact. (3) A *KV cache resizer* dynamically adjusts KV cache capacity to accommodate the post-reconfiguration layer placement and reclaims memory from removed layers; when shrinking, it performs cache compaction to mitigate fragmentation and restore usable free space.

At the *attention kernel level*, existing PagedAttention implementations pre-allocate contiguous KV cache regions per layer, which prevents efficient resizing during reconfiguration. FlexiServe extends PagedAttention with a block-level KV cache pre-allocation strategy and employs direct block-table addressing to reduce the overhead of indirect block-pointer lookups. This design enables block-level KV cache resizing and compaction while preserving performance comparable to the original PagedAttention kernel.

Overall, the Reconfiguration Coordinator makes global, SLO-aware decisions and translates them into concrete reconfiguration plans; Reconfiguration Workers execute these plans asynchronously on each GPU while inference workers continue serving requests; and kernel-level KV cache support enables efficient resizing and compaction without sacrificing attention performance. By coordinating reconfiguration across all three layers, FlexiServe minimizes model downtime during PP reconfiguration and bounds its impact on throughput, TTFT, and TPOT, enabling practical and efficient dynamic reconfiguration for LLM serving under heterogeneous and evolving workloads.

5 Reconfiguration Coordinator

The *Reconfiguration Coordinator* is the central coordination component that orchestrates PP reconfiguration in FlexiServe. It implements a *PP reconfiguration protocol* that specifies how a target PP configuration is realized through a sequence of coordinated actions across GPUs. Rather than treating reconfiguration as a monolithic operation, the protocol decomposes PP reconfiguration into a set of well-defined *primitives*, each capturing a fundamental reconfiguration action. These primitives are invoked via collective RPCs and executed by GPU-level Reconfiguration Worker in a coordinated manner, enabling reconfiguration to proceed concurrently with inference while tightly controlling resource usage and interference.

In this section, we first provide an end-to-end overview of the PP reconfiguration protocol. We then describe the core reconfiguration primitives defined by the protocol and

their semantics. Finally, we present a reconfiguration algorithm that composes these primitives to safely and efficiently perform online PP reconfiguration at runtime.

5.1 General Reconfiguration Procedure

Figure X illustrates a canonical PP reconfiguration workflow. We denote a PP configuration as an ordered tuple of layer ranges assigned to each GPU. For clarity, we consider a simplified setup with two GPUs (rank 0 and rank 1) serving a six-layer model. The initial configuration is $C_A = \langle [1, 3], [4, 6] \rangle$, which evenly partitions the model across the two GPUs. The system then transitions to a target configuration $C_B = \langle [1, 2], [3, 6] \rangle$, requiring one layer to be migrated from rank 0 to rank 1.

At the outset, both GPUs have most of their memory occupied by model weights and KV caches. To create space for the incoming layer, rank 1 first resizes its KV cache, freeing sufficient memory to accommodate the additional layer weights and KV state. Once resizing completes, rank 1 begins loading the new layer weights, while rank 0 concurrently streams the corresponding KV cache. Importantly, throughout this migration phase, inference continues under the original configuration C_A , meaning that both GPUs still execute three layers. As a result, the migrating layer on rank 0 continues to generate new KV entries, which are incrementally synchronized to rank 1 via streaming KV transfer.

Meanwhile, the reconfiguration coordinator monitors KV cache synchronization progress by tracking the latest KV index produced on rank 0 and the highest index received on rank 1. Once the lag falls below a predefined threshold, the reconfigurator inserts a synchronization point, performs a final KV cache synchronization, and atomically switches the system to the new configuration C_B . After the switch, rank 0 deletes the migrated layer and releases its weights and KV cache. Then the KV cache is resized again to reclaim freed memory.

Several challenges arise during PP reconfiguration. First, migration requires resizing the KV cache to free memory for newly assigned layers; the system must ensure that the target PP configuration is feasible and can be accommodated on destination GPUs without interrupting ongoing inference. Second, when scaling beyond a two-GPU setting, the system must correctly and concurrently coordinate the sequence of stateful, latency-sensitive actions illustrated in Figure X and commit the configuration change consistently across all devices.

To address these challenges, we design a PP reconfiguration protocol that formalizes the required primitives, their coordination logic, and how a PP configuration change can be safely committed. We then present an algorithm that composes these primitives to efficiently transition between PP configurations at runtime.

5.2 Pipeline Parallelism Reconfiguration Protocol

The PP reconfiguration protocol exposes a set of *primitives* invoked by the Reconfiguration Coordinator to orchestrate coordinated reconfiguration across GPUs. Based on their invocation semantics, these primitives fall into two categories: *Collective Primitives (CP)* and *Synchronization Primitives (SP)*. As illustrated in Figure X, CP are issued by the Reconfiguration Coordinator via collective RPCs and executed by all Reconfiguration Workers. Each GPU performs the specified local actions on its assigned layer ranges independently, without a globally enforced execution order, allowing CP to proceed asynchronously across devices.

In contrast, SP are embedded into the metadata of *scheduled inference batches* and are executed only when all GPUs are processing the *same batch*. Embedding SP at the batch level guarantees that every GPU observes the synchronization primitive at an identical pipeline boundary and applies it in pipeline order. This ensures that global state transitions—such as committing a new PP configuration—occur atomically with respect to a batch, preserving pipeline consistency and avoiding inference interruption.

To uniformly represent per-GPU actions in collective primitives, we use a *layer assignment map* $\mathcal{M}$. $\mathcal{M}$ maps GPU ranks to sets of layer ranges, i.e., $\mathcal{M} : r \mapsto \{[l_{1,lo}^{(r)}, l_{1,hi}^{(r)}], \dots\}$, where $\mathcal{M}(r)$ specifies the layer ranges on which GPU r should operate for a given primitive. For primitives involving data transfer across GPUs, we use a directional assignment map $\mathcal{M}_{\text{mig}}$, where $\mathcal{M}_{\text{mig}}(r_s, r_d)$ specifies the set of layer ranges transfer from source GPU r_s to destination GPU r_d .

The primitives are defined as follows:

- **COLLECTIVE::COMPACTKV(B)**: Compacts the KV cache on each GPU to B blocks, reclaiming fragmented space required for subsequent KV resizing operation.
- **COLLECTIVE::RESIZEKV(B)**: Resizes the KV cache capacity on each GPU to B blocks
- **COLLECTIVE::ADDLAYERWEIGHTS($\mathcal{M}_{\text{add}}$)**: Instructs each Reconfiguration Worker r to asynchronously load the weights for the layer ranges specified in $\mathcal{M}_{\text{add}}(r)$.
- **COLLECTIVE::STARTKVMIGRATION($\mathcal{M}_{\text{mig}}$)**: Initiates asynchronous KV cache migration according to $\mathcal{M}_{\text{mig}}$, where $\mathcal{M}_{\text{mig}}(r_s, r_d)$ specifies the layer ranges whose KV states should be streamed from source GPU r_s to destination GPU r_d .
- **SYNC::SYNCANDCOMMIT($\mathcal{M}_{\text{del}}, B_{\text{new}}$)**: Performs a synchronized state transition that atomically commits the new PP configuration. After the commit, each Reconfiguration Worker r deletes the layer weights and KV cache entries specified in $\mathcal{M}_{\text{del}}(r)$, and resizes the KV cache to B_{new} blocks to reclaim freed memory.

With the above primitives, we present our reconfiguration algorithm. Table ?? summarizes the notation used in Algorithm ??.

Algorithm 1: ASYNC PP RECONFIGURATION

The algorithm transitions from C_{cur} to C_{tgt} through an *intermediate configuration* C_{int} in which both old and new layers coexist, enabling continuous inference throughout the migration.

```

1 Function MaxBlocks( $r, L$ ):
2   return  $\lfloor (M_r \cdot u - L \cdot W) / (L \cdot P) \rfloor$ 
Input:  $C_{\text{cur}} = \{(s_r, e_r)\}_{r=0}^{R-1}$ : current PP config, rank  $r$  owns layers
          $[s_r, e_r]$ ;  $C_{\text{tgt}} = \{(s'_r, e'_r)\}_{r=0}^{R-1}$ : target PP config;  $\tau$ : convergence
         threshold (tokens)
Output: true if reconfiguration succeeds; false if infeasible

  > Phase 1: Feasibility assessment & intermediate
  configuration
3  $C_{\text{int}} \leftarrow C_{\text{cur}}$ ;  $M_{\text{add}} \leftarrow \emptyset$ 
4 foreach rank  $r \in [0, R)$  do           > build intermediate config  $C_{\text{int}}$ 
5    $C_{\text{int}}[r] \leftarrow C_{\text{cur}}[r] \cup C_{\text{tgt}}[r]$    > new layers rank  $r$  must add
6    $B_{\text{max}}(r) \leftarrow \text{MaxBlocks}(r, |C_{\text{int}}[r]|)$ 
7 end
8  $B_{\text{shrink}} \leftarrow \min_r B_{\text{max}}(r)$ 
9  $B_{\text{used}} \leftarrow$  current number of KV blocks in use
10 if  $B_{\text{used}} > B_{\text{shrink}}$  then
11   return false > insufficient memory for reconfiguration, abort
12 end

  > Phase 2: KV Compaction & Resizing
13 if  $B_{\text{shrink}} < B_{\text{cur}}$  then
14   Collective::CompactKV( $B_{\text{shrink}}$ )
15   Collective::ResizeKV( $B_{\text{shrink}}$ )
16 end

  > Phase 3: Asynchronous weight loading and KV Cache
  Migration(non-blocking, concurrent with inference)
17 Collective::AddLayerWeights( $M_{\text{add}}$ )
18  $M_{\text{mig}} \leftarrow \emptyset$ 
19 foreach ( $r_{\text{dst}}, A_{r_{\text{dst}}}$ )  $\in M_{\text{add}}$  do
20   foreach layer  $\ell \in A_{r_{\text{dst}}}$  do
21      $r_{\text{src}} \leftarrow$  rank owning layer  $\ell$  in  $C_{\text{cur}}$ 
22      $M_{\text{mig}}[r_{\text{src}}][r_{\text{dst}}] \leftarrow M_{\text{mig}}[r_{\text{src}}][r_{\text{dst}}] \cup \{\ell\}$ 
23   end
24 end
25 Collective::StartKVMigration( $M_{\text{mig}}$ )

  > Phase 4: Convergence monitoring
26 repeat
27    $T_{\text{sched}} \leftarrow \text{getLastScheduledTokenIndex}()$ 
28    $T_{\text{applied}}[] \leftarrow \text{Collective::GetLastSyncedTokenIndex}()$ 
29    $\text{allDone} \leftarrow \text{true}$ 
30   foreach rank  $r \in \text{dom}(M_{\text{add}})$  do
31     if  $T_{\text{sched}} - T_{\text{applied}}[r] \geq \tau$  then
32        $\text{allDone} \leftarrow \text{false}$ 
33     break
34   end
35 end
36 if  $\text{allDone}$  then break
37  $\text{sleep}(\Delta t)$ 
38 until

  > Phase 5: Commit PP Configuration Change
39
40 foreach rank  $r \in [0, R)$  do           > compute layers to delete from  $C_{\text{int}}$ 
41    $M_{\text{del}}(r) \leftarrow C_{\text{int}}[r] \setminus C_{\text{tgt}}[r]$ 
42    $B_{\text{new}}(r) \leftarrow \text{MaxBlocks}(r, |C_{\text{tgt}}[r]|)$ 
43 end
44  $B_{\text{new}} \leftarrow \min_r B_{\text{new}}(r)$ 
45 Sync::SyncAndCommit( $M_{\text{del}}, B_{\text{new}}$ )   > commit new config, then
   delete old layers & resize KV
46 return true

```

Table 1. Notation used in Algorithm ??.

Symbol	Description
$C_{\text{cur}}, C_{\text{tgt}}, C_{\text{int}}$	$r \mapsto \{\ell_1, \dots\}$: rank $\rightarrow$ layer set (current / target / intermediate)
$M_{\text{add}}, M_{\text{del}}$	$r \mapsto \{\ell_1, \dots\}$: rank $\rightarrow$ layers to add / delete
M_{mig}	$r_{\text{src}} \mapsto (r_{\text{dst}} \mapsto \{\ell_1, \dots\})$: layers to migrate
$B_{\text{max}}(r)$	Max KV blocks rank r can hold
$B_{\text{cur}}, B_{\text{new}}$	Current / post-reconfiguration KV block budget
M_r	Total GPU memory on rank r
u	KV cache utilization ratio, $u \in (0, 1]$
W	Weight memory per layer
P	KV page (block) size
τ	Convergence threshold (tokens)
Δt	Polling interval

The key idea behind Algorithm ?? is to transition from C_{cur} to C_{tgt} through an *intermediate configuration* C_{int} in which each rank holds the union of its current and target layer sets, allowing inference to continue uninterrupted throughout the migration. The algorithm proceeds in five phases.

Phase 1: Feasibility assessment. For each rank r , the algorithm computes the intermediate layer set $C_{\text{int}}[r] = C_{\text{cur}}[r] \cup C_{\text{tgt}}[r]$ and derives the maximum KV block capacity $B_{\text{max}}(r)$ under C_{int} . The global KV budget is set to $B_{\text{shrink}} = \min_r B_{\text{max}}(r)$. If the number of KV blocks currently in use exceeds B_{shrink} , the reconfiguration is deemed infeasible and the algorithm aborts.

Phase 2: KV compaction and resizing. If $B_{\text{shrink}} < B_{\text{cur}}$, the coordinator invokes COMPACTKV followed by RESIZEKV to shrink the KV cache on every rank, freeing memory for the additional layer weights that will be loaded in the next phase.

Phase 3: Asynchronous weight loading and KV migration. The coordinator issues ADDLAYERWEIGHTS to begin loading new layer weights on the destination ranks. Concurrently, it constructs the migration map M_{mig} —mapping each source rank to its destination ranks and the associated layers—and invokes STARTKVMIGRATION to begin streaming KV states. Both operations proceed asynchronously alongside ongoing inference.

Phase 4: Convergence monitoring. The coordinator polls the gap between the last scheduled token index T_{sched} and the last synchronized token index $T_{\text{applied}}[r]$ on each destination rank. Once this gap falls below the convergence threshold τ for all ranks in $\text{dom}(M_{\text{add}})$, the KV state is considered sufficiently up-to-date and the algorithm proceeds to commit.

Phase 5: Commit. The coordinator computes the deletion map M_{del} (layers in C_{int} no longer needed under C_{tgt}) and the post-reconfiguration KV budget B_{new} . It then issues SYNCANDCOMMIT, which atomically switches the pipeline to C_{tgt} , deletes obsolete layer weights and KV entries, and resizes the KV cache to B_{new} to reclaim freed memory.

6 Reconfiguration Worker

Temporary page!

ℒ_TX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because ℒ_TX now knows how many pages to expect for this document.